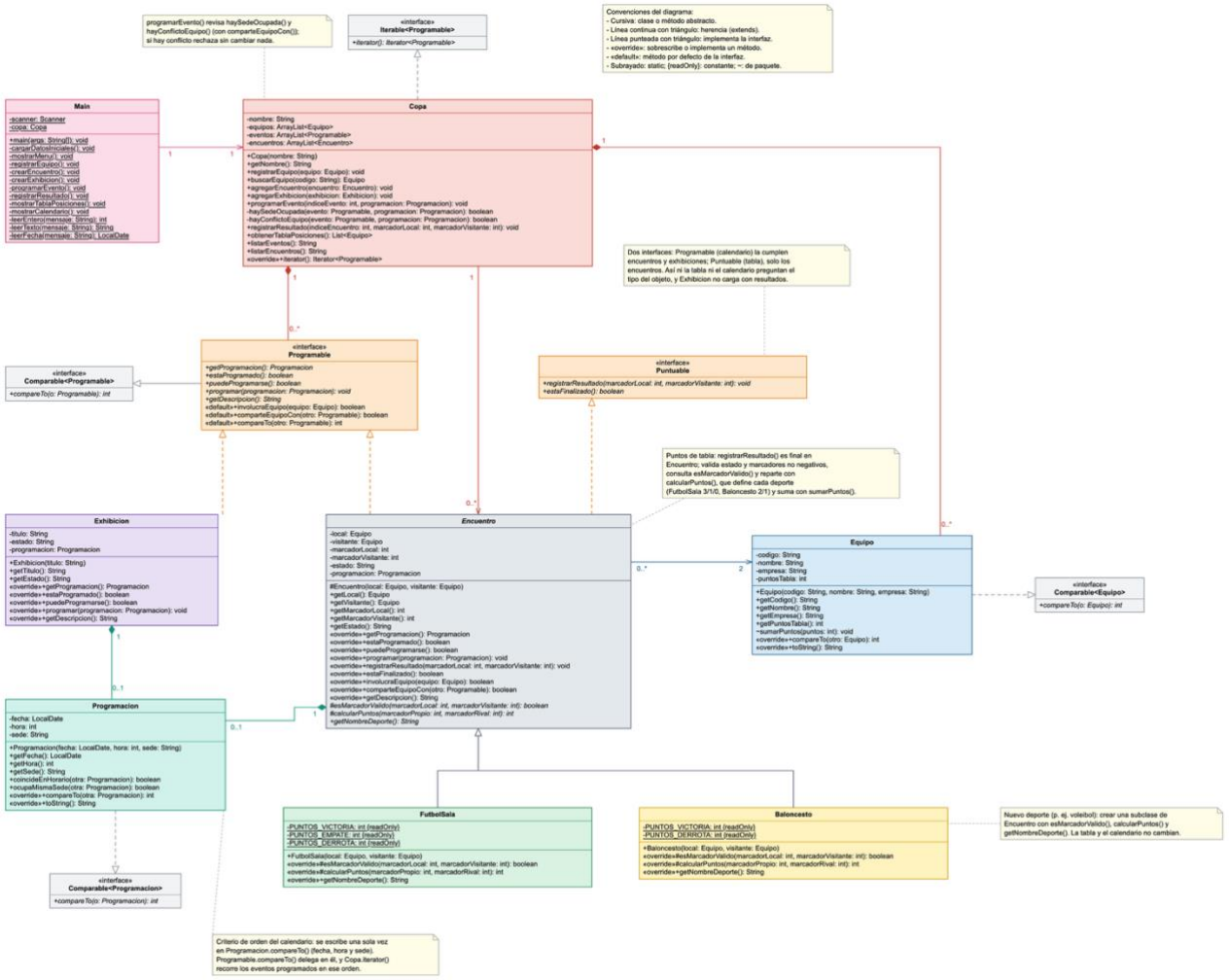
Nery Javier de la Cruz Huinil

261233



Propiedades					
#	**tipoDato**	**nombre**	**visibilidad**	**motivo**	**Clase**
1	LocalDate	fecha	private	Día en que se realiza el evento; primer criterio del orden cronológico.	Programacion
2	int	hora	private	Hora de inicio completa (0-23); segundo criterio del orden.	Programacion
3	String	sede	private	Lugar del evento; tercer criterio del orden y base del conflicto de sede.	Programacion
4	String	codigo	private	Identificador único del equipo; no puede repetirse ni estar vacío.	Equipo
5	String	nombre	private	Nombre del equipo; se usa para desempatar en la tabla.	Equipo
6	String	empresa	private	Empresa a la que representa el equipo.	Equipo
7	int	puntosTabla	private	Puntos de tabla acumulados; distintos de los goles o puntos del marcador.	Equipo
8	Equipo	local	private	Primer equipo del encuentro; se recibe ya creado.	Encuentro (abstracta)
9	Equipo	visitante	private	Segundo equipo; no puede ser el mismo que el local.	Encuentro (abstracta)

Propiedades					
#	**tipoDato**	**nombre**	**visibilidad**	**motivo**	**Clase**
10	int	marcadorLocal	private	Goles o puntos anotados por el equipo local.	Encuentro (abstracta)
11	int	marcadorVisitante	private	Goles o puntos anotados por el equipo visitante.	Encuentro (abstracta)
12	String	estado	private	Sin programar, Programado o Finalizado; controla qué operaciones son válidas.	Encuentro (abstracta)
13	Programacion	programacion	private	Fecha, hora y sede del encuentro; null mientras no esté programado.	Encuentro (abstracta)
14	int	PUNTOS_VICTORIA	private static final	Constante con los 3 puntos de tabla por ganar; evita números mágicos.	FutbolSala
15	int	PUNTOS_EMPATE	private static final	Constante con el 1 punto de tabla por empatar.	FutbolSala
16	int	PUNTOS_DERROTA	private static final	Constante con los 0 puntos de tabla por perder.	FutbolSala
17	int	PUNTOS_VICTORIA	private static final	Constante con los 2 puntos de tabla por ganar.	Baloncesto
18	int	PUNTOS_DERROTA	private static final	Constante con el 1 punto de tabla por perder.	Baloncesto
19	String	titulo	private	Nombre de la exhibición (por ejemplo, "Clínica de baloncesto").	Exhibicion

Propiedades					
#	**tipoDato**	**nombre**	**visibilidad**	**motivo**	**Clase**
20	String	estado	private	Sin programar o Programada.	Exhibicion
21	Programacion	programacion	private	Fecha, hora y sede de la exhibición; null mientras no esté programada.	Exhibicion
22	String	nombre	private	Nombre de la copa para mostrar en consola.	Copa
23	ArrayList<Equipo>	equipos	private	Guarda los equipos registrados; oculto para no exponer la estructura interna.	Copa
24	ArrayList<Programable>	eventos	private	Guarda encuentros y exhibiciones juntos para armar el calendario.	Copa
25	ArrayList<Encuentro>	encuentros	private	Referencias a los encuentros para registrar resultados sin preguntar por el tipo del objeto.	Copa
26	Scanner	scanner	private static	Lector único de la entrada del usuario para el menú.	Main
27	Copa	copa	private static	Copa con la que trabaja el menú durante la ejecución.	Main

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
1	Programacion	getProgramacion()	public abstract	Devuelve fecha, hora y sede del evento; contrato común del calendario.	Programable (interfaz)
2	boolean	estaProgramado()	public abstract	Indica si el evento ya tiene fecha para mostrarse en el calendario.	Programable (interfaz)
3	boolean	puedeProgramarse()	public abstract	Indica si el evento admite programarse o reprogramarse (un encuentro finalizado no).	Programable (interfaz)
4	void	programar(Programacion programacion)	public abstract	Asigna o cambia la fecha, hora y sede del evento.	Programable (interfaz)
5	String	getDescripcion()	public abstract	Texto que el calendario muestra sin conocer el tipo de evento.	Programable (interfaz)
6	boolean	involucraEquipo(Equipo equipo)	public default	Devuelve false por defecto; solo los encuentros tienen equipos.	Programable (interfaz)
7	boolean	comparteEquipoCon(Programable otro)	public default	Devuelve false por defecto; permite	Programable (interfaz)

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
				detectar conflictos de equipo.	
8	int	compareTo(Programable otro)	public default	Implementa Comparable delegando en Programacion; el criterio de orden se escribe una sola vez.	Programable (interfaz)
9	void	registrarResultado(int marcadorLocal, int marcadorVisitante)	public abstract	Recibe el marcador y reparte los puntos de tabla a los dos equipos.	Puntuable (interfaz)
10	boolean	estaFinalizado()	public abstract	Indica si ya tiene resultado y no admite otro.	Puntuable (interfaz)
11	—	Programacion(LocalDate fecha, int hora, String sede)	public	Constructor: valida hora 0-23 y sede no vacía antes de crear la programación.	Programacion
12	LocalDate	getFecha()	public	Lectura de la fecha del evento.	Programacion
13	int	getHora()	public	Lectura de la hora de inicio.	Programacion
14	String	getSede()	public	Lectura de la sede.	Programacion
15	boolean	coincideEnHorario(Programacion otra)	public	Indica si dos programaciones	Programacion

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
				tienen la misma fecha y hora.	
16	boolean	ocupaMismaSede(Programacion otra)	public	Indica si coinciden en fecha, hora y sede; base del conflicto de sede.	Programacion
17	int	compareTo(Programacion otra)	public	@Override de Comparable: ordena por fecha, luego hora y luego nombre de sede.	Programacion
18	String	toString()	public	@Override: muestra fecha, hora y sede en formato legible.	Programacion
19	—	Equipo(String codigo, String nombre, String empresa)	public	Constructor: valida que los datos no estén vacíos e inicia puntosTabla en 0.	Equipo
20	String	getCodigo()	public	Lectura del código para evitar duplicados y buscar equipos.	Equipo
21	String	getNombre()	public	Lectura del nombre para mostrar y desempatar.	Equipo

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
22	String	getEmpresa()	public	Lectura de la empresa representada.	Equipo
23	int	getPuntosTabla()	public	Lectura de los puntos de tabla para ordenar la tabla de posiciones.	Equipo
24	void	sumarPuntos(int puntos)	default (paquete)	Suma puntos de tabla; solo lo usan los encuentros para proteger el estado.	Equipo
25	int	compareTo(Equipo otro)	public	@Override de Comparable: ordena por puntos de mayor a menor y, si empatan, por nombre.	Equipo
26	String	toString()	public	@Override: muestra la fila del equipo en la tabla de posiciones.	Equipo
27	—	Encuentro(Equipo local, Equipo visitante)	protected	Constructor: valida que los equipos sean distintos e inicia en Sin programar; lo llaman las subclases con super().	Encuentro (abstracta)

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
28	Equipo	getLocal()	public	Lectura del equipo local.	Encuentro (abstracta)
29	Equipo	getVisitante()	public	Lectura del equipo visitante.	Encuentro (abstracta)
30	int	getMarcadorLocal()	public	Lectura del marcador del equipo local.	Encuentro (abstracta)
31	int	getMarcadorVisitante()	public	Lectura del marcador del equipo visitante.	Encuentro (abstracta)
32	String	getEstado()	public	Lectura del estado actual del encuentro.	Encuentro (abstracta)
33	Programacion	getProgramacion()	public	@Override de Programable: devuelve la programación del encuentro.	Encuentro (abstracta)
34	boolean	estaProgramado()	public	@Override de Programable: true si tiene programación asignada.	Encuentro (abstracta)
35	boolean	puedeProgramarse()	public	@Override de Programable: true mientras el encuentro no esté finalizado.	Encuentro (abstracta)
36	void	programar(Programacion programacion)	public	@Override de Programable: asigna la programación y	Encuentro (abstracta)

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
				cambia el estado a Programado.	
37	void	registrarResultado(int marcadorLocal, int marcadorVisitante)	public final	@Override de Puntuable: valida estado, valores no negativos y esMarcadorValido(); luego suma puntos y finaliza.	Encuentro (abstracta)
38	boolean	estaFinalizado()	public	@Override de Puntuable: true si el estado es Finalizado.	Encuentro (abstracta)
39	boolean	involucraEquipo(Equipo equipo)	public	@Override de Programable: true si el equipo es el local o el visitante.	Encuentro (abstracta)
40	boolean	comparteEquipoCon(Programable otro)	public	@Override de Programable: revisa si el otro evento involucra a alguno de sus equipos.	Encuentro (abstracta)
41	String	getDescripcion()	public	@Override de Programable: muestra deporte, equipos y marcador si existe.	Encuentro (abstracta)

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
42	boolean	esMarcadorValido(int marcadorLocal, int marcadorVisitante)	protected abstract	Queda sin resolver: cada deporte define qué marcador acepta.	Encuentro (abstracta)
43	int	calcularPuntos(int marcadorPropio, int marcadorRival)	protected abstract	Queda sin resolver: cada deporte define cuántos puntos de tabla reparte.	Encuentro (abstracta)
44	String	getNombreDeporte()	public abstract	Cada subclase devuelve el nombre de su deporte para mostrarlo.	Encuentro (abstracta)
45	—	FutbolSala(Equipo local, Equipo visitante)	public	Constructor: envía los equipos a Encuentro mediante super().	FutbolSala
46	boolean	esMarcadorValido(int marcadorLocal, int marcadorVisitante)	protected	@Override: acepta cualquier marcador, incluido el empate.	FutbolSala
47	int	calcularPuntos(int marcadorPropio, int marcadorRival)	protected	@Override: devuelve 3 por victoria, 1 por empate y 0 por derrota.	FutbolSala
48	String	getNombreDeporte()	public	@Override: devuelve "Fútbol sala".	FutbolSala
49	—	Baloncesto(Equipo local, Equipo visitante)	public	Constructor: envía los equipos a Encuentro mediante super().	Baloncesto

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
50	boolean	esMarcadorValido(int marcadorLocal, int marcadorVisitante)	protected	@Override: rechaza marcadores empatados.	Baloncesto
51	int	calcularPuntos(int marcadorPropio, int marcadorRival)	protected	@Override: devuelve 2 por victoria y 1 por derrota.	Baloncesto
52	String	getNombreDeporte()	public	@Override: devuelve "Baloncesto".	Baloncesto
53	—	Exhibicion(String titulo)	public	Constructor: valida que el título no esté vacío e inicia en Sin programar.	Exhibicion
54	String	getTitulo()	public	Lectura del título de la exhibición.	Exhibicion
55	String	getEstado()	public	Lectura del estado actual.	Exhibicion
56	Programacion	getProgramacion()	public	@Override de Programable: devuelve la programación de la exhibición.	Exhibicion
57	boolean	estaProgramado()	public	@Override de Programable: true si ya tiene programación.	Exhibicion
58	boolean	puedeProgramarse()	public	@Override de Programable: siempre	Exhibicion

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
				true, porque no tiene resultado.	
59	void	programar(Programacion programacion)	public	@Override de Programable: asigna la programación y cambia a Programada.	Exhibicion
60	String	getDescripcion()	public	@Override de Programable: muestra "Exhibición: " y el título.	Exhibicion
61	—	Copa(String nombre)	public	Constructor: recibe el nombre e inicializa las listas vacías.	Copa
62	String	getNombre()	public	Lectura del nombre de la copa.	Copa
63	void	registrarEquipo(Equipo equipo)	public	Agrega un equipo; rechaza códigos repetidos sin modificar el estado.	Copa
64	Equipo	buscarEquipo(String codigo)	public	Devuelve el equipo con ese código o null si no existe.	Copa
65	void	agregarEncuentro(Encuentro encuentro)	public	Guarda el encuentro en eventos y en encuentros.	Copa

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
66	void	agregarExhibicion(Exhibicion exhibicion)	public	Guarda la exhibición en la lista de eventos.	Copa
67	void	programarEvento(int indiceEvento, Programacion programacion)	public	Valida conflictos de sede y de equipo; si no hay, programa o reprograma el evento.	Copa
68	boolean	haySedeOcupada(Programable evento, Programacion programacion)	private	Revisa si otro evento usa la misma sede en esa fecha y hora.	Copa
69	boolean	hayConflictoEquipo(Programable evento, Programacion programacion)	private	Revisa si algún equipo ya tiene otro encuentro en esa fecha y hora.	Copa
70	void	registrarResultado(int indiceEncuentro, int marcadorLocal, int marcadorVisitante)	public	Delega el resultado al encuentro a través de la interfaz Puntuable.	Copa
71	List<Equipo>	obtenerTablaPosiciones()	public	Devuelve una copia de los equipos ordenada con compareTo de Equipo.	Copa
72	String	listarEventos()	public	Devuelve los eventos numerados para que el usuario elija en el menú.	Copa

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
73	String	listarEncuentros()	public	Devuelve los encuentros numerados para registrar resultados.	Copa
74	Iterator<Programable>	iterator()	public	@Override de Iterable: recorre los eventos programados en orden cronológico sin exponer la lista interna.	Copa
75	void	main(String[] args)	public static	Punto de entrada: carga los datos iniciales y muestra el menú hasta que el usuario salga.	Main
76	void	cargarDatosIniciales()	private static	Crea equipos, encuentros con resultado y una exhibición programada desde el código.	Main
77	void	mostrarMenu()	private static	Muestra las opciones y llama a la operación elegida.	Main
78	void	registrarEquipo()	private static	Pide código, nombre y empresa, y registra el equipo en la copa.	Main

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
79	void	crearEncuentro()	private static	Pide deporte y equipos, y crea un FutbolSala o un Baloncesto.	Main
80	void	crearExhibicion()	private static	Pide el título y agrega la exhibición a la copa.	Main
81	void	programarEvento()	private static	Pide evento, fecha, hora y sede, y programa o reprograma.	Main
82	void	registrarResultado()	private static	Pide el encuentro y el marcador, y muestra el mensaje si se rechaza.	Main
83	void	mostrarTablaPosiciones()	private static	Imprime la tabla de posiciones ordenada.	Main
84	void	mostrarCalendario()	private static	Recorre la copa con for-each e imprime cada evento.	Main
85	int	leerEntero(String mensaje)	private static	Pide un número hasta que sea válido; evita que el programa termine por mal formato.	Main

Métodos					
#	**retorno**	**nombre**	**visibilidad**	**motivo**	**Clase**
86	String	leerTexto(String mensaje)	private static	Pide un texto y rechaza entradas vacías.	Main
87	LocalDate	leerFecha(String mensaje)	private static	Pide una fecha en formato aaaa-mm-dd y repite si el formato es incorrecto.	Main